Code generation from scratch

HW2 experiment report - work in progress
Andrei Vorsin | 16 October 2026

Current result

Verified and scored checkpoints reach 180,000 updates. Ten initial checkpoints, eight first-continuation checkpoints and 6 longer-continuation checkpoints were saved in Google Drive and scored by the course bot. The best public result is **59.90% (575/960)**. The bot reported rank 9 of 16 at 2026-10-16T14:48:00+03:00. Its exact scored output is frequency-screen-control-073024.jsonl. This is the current best result across our decoding experiments and uses strict greedy decoding. Later sections distinguish historical settings. T4 timing remains deferred. No private result is available.

Task and rules

Train an encoder-decoder Transformer from random weights using only the supplied SlavaYus/tinysms_qa_12m training split. Generate one Python program per test question. All 1,919 programs must be generated within 180 seconds on one NVIDIA T4. No pretrained weights, outside examples, model distillation, or agent-written target programs are used.

Data and architecture

The allowed dataset contains 11,838,003 question/code pairs in 60 training files. The test file contains questions only. A byte-level BPE tokenizer with 8,192 tokens was fitted on shard 0 after reserving its first 1,024 rows. The initial baseline has 9,967,360 parameters: four encoder layers, four decoder layers, width 256, feed-forward width 768, four attention heads, gated GELU and dropout 0.1. It uses the T5 architecture with randomly initialized weights. The current selected larger model has 42,481,152 parameters, six encoder layers, four decoder layers, width 512 and feed-forward width 1536; the full run stopped at 70k, the matched loss trials each added 2k updates, and the current best adds 1,024 control updates.

Training uses AdamW, 500 warmup updates, cosine decay to 10% of the initial rate, weight decay 0.01 and gradient clipping at 1.0. The first 20,000 updates used batch 64 and peak learning rate 0.001. The first continuation retained optimizer moments, used batch 128 and restarted the schedule at peak rate 0.0005. The 60k-to-180k continuation retains batch 128 and optimizer moments, with a new schedule peaking at 0.0003. Input and target limits are 128 and 192 tokens. Padding tokens do not contribute to the loss. Model and optimizer states are saved regularly.

Validation correction

Earlier runs excluded the first 1,024 row positions but did not exclude copies of their questions. An audit found 24 later rows in shard 0 with the same question text. Historical validation was therefore not fully independent. Future streams exclude every reserved question text across all shards. Rows 0-255 select checkpoints; rows 256-511 provide separate diagnostics. The earlier tokenizer may have seen duplicate question text.

Quality experiments

Historical runs

Experiment	Evidence	Interpretation
Local MPS / short T4 checks	One local update; ten T4 updates; later T4 pilot reached 4,000 updates, loss 1.6469.	Pipeline checks. The longer pilot checkpoint was not recovered.
Sequential shards, 20,000 updates	Final held-out token loss 1.1213; best logged 0.9207 at step 12,500.	Best weights were not retained; last weights only.
Rotate after 1,000 mini-batches, 20,000 updates	Final loss 0.9815; saved best 0.8636 at step 18,500.	Selected historical checkpoint. Held-out exact code match 2/256; public score 18.85%.

The historical comparison supports broader data coverage by token loss, but the validation-copy issue limits that conclusion. Token loss scores the next token with the reference prefix supplied; it does not show whether a generated program computes the right answer.

Audit before the new pilots

All 1,024 held-out programs round-trip through the tokenizer exactly with cleanup disabled. Only 2 held-out questions and 3 of 1,919 test questions exceed the input limit; 5 held-out programs exceed the target limit. Truncation and tokenizer corruption do not explain most failures in this audit. Samples of 1,000 rows from shards 0, 15, 30, 45 and 55 have mean code lengths 65.96, 62.56, 93.17, 61.84 and 118.95 tokens. This suggests that sequential training encounters different styles for extended periods.

Matched sampling pilot

Hypothesis: mixing original examples across shards improves held-out numeric agreement. Two fresh runs use identical initialization, model, seed, BF16 precision, batch 64, accumulation 1, grouping window 32 and 6,000 updates. Both exclude validation question texts. The control rotates after 1,000 batches. The treatment mixes up to 4,096 randomly selected original rows from every shard and resamples each epoch. Sampling is the experimental variable. Comparisons with older runs also change batch shape and validation exclusion.

Run	Best token loss	Held-out numeric agreement
control	1.6012	14/245
mixed	1.4793	16/245
mixed_lr001	1.1409	27/245

Numeric evaluation compares solve() outputs of generated and supplied held-out programs, using limited separate processes and restricted imports. Only finite numeric reference results are usable. The old saved model agrees on 50/245 usable references (20.4%). Reference errors and runner restrictions can affect this measure. Numeric outputs are never used as training targets.

Completed Drive-backed run

All ten checkpoints retained

The mixed sampler completed 20,000 updates with batch 64, BF16 training and a peak learning rate of 0.001. Source commit: 1a23e45b1138c3dba7c40befeae6d3337153df98. Training plus periodic validation, generation and Drive saves took about 12 minutes on an NVIDIA RTX PRO 6000 Blackwell Server Edition. This is not a T4 timing measurement.

Step	Validation token loss	Public correct / 960	Public score
2,000	1.8064	31	3.23%
4,000	1.2130	49	5.10%
6,000	1.0967	93	9.69%
8,000	1.0020	103	10.73%
10,000	0.9396	141	14.69%
12,000	0.8848	133	13.85%
14,000	0.8540	163	16.98%
16,000	0.8219	182	18.96%
18,000	0.8076	192	20.00%
20,000	0.7952	195	20.31%

Every checkpoint was written to Drive with optimizer state and a completion manifest before predictions were generated. Each checkpoint has 1,919 exact model outputs, matching CSV and JSONL files, validation metrics and provenance hashes. The files were exported from Drive, checked locally and then submitted to the bot. Colab was disconnected before the bot evaluations finished.

Drive location: MyDrive/HSE_NLP_HW2/runs/mixed-lr001-20k-drive-v1. The repository keeps exported prediction evidence and individual submission receipts. Weights stay in Drive. The final validation loss is lower than in the short pilots, but scores across checkpoints show that lower token loss does not guarantee higher program accuracy.

Longer training and GPU use

Eight more checkpoints, all scored

The continuation added 40,000 updates and 5,120,000 example presentations in 1,957.2 seconds (32.6 minutes), including validation, generation and Drive saves. It resumed our own step-20,000 model. Source commit: a411a64f8afe61e3a2b337af338c0ea6c19c9c15. No new target programs or external model weights were introduced.

Step	Validation token loss	Public correct / 960	Public score
25,000	0.7647	194	20.21%
30,000	0.7120	227	23.65%
35,000	0.6817	231	24.06%
40,000	0.6451	273	28.44%
45,000	0.6058	314	32.71%
50,000	0.5857	343	35.73%
55,000	0.5701	353	36.77%
60,000	0.5652	362	37.71%

Batch size was chosen from two matched 16,384-example probes with reversed order. Median throughput was 2,100 examples/s at batch 64; 2,279 at 128; 2,114 at 256; and 1,950 at 512. Batch 128 was 8.5% faster than 64, with peak allocated memory 7.75 GiB. Larger batches raised utilization but reduced useful throughput.

Across the continuation, 1,972 one-second GPU samples averaged 60.38% utilization, with median 78%, 16.23% zero samples and peak used memory 10,397 MiB. These include checkpoint saving and generation. CPU preparation occupied about 41% of the batch-128 probe, suggesting a remaining bottleneck. The GPU was released and Colab showed no active sessions at 19:07 +03:00.

Drive location: MyDrive/HSE_NLP_HW2/runs/continue-20k-60k-b128-v1. Every checkpoint has exact 1,919-row outputs in official test order, provenance hashes and a bot receipt. The step-60,000 Blackwell generation took 9.07 seconds; its T4 timing remains unverified.

Faster input preparation

Matched CPU and GPU checks

CPU preparation compared the original batch path, reuse of window tokenization, and a bounded two-batch preparation queue. All three produced identical input IDs, masks and labels for 32,768 examples and 2,419,826 target tokens. Preparation took 18.72, 6.38 and 5.98 seconds respectively. Eighteen local checks covered truncation, duplicates, partial windows, literal special tokens and producer failures. Synthetic fixtures stayed confined to tests.

Input path	Median examples / second
legacy	2,931.17
encoded	3,483.24
encoded-prefetch	3,692.80

Two GPU rounds used reversed variant order, the same checkpoint, seed, examples, precision and batch size. The selected two-batch queue improved median throughput by 25.98% over the original path, with peak allocated tensor memory 7.75 GiB. The tokenizer used by the producer is separate from the validation tokenizer. This is a matched speed result; utilization alone does not measure useful throughput.

Longer continuation: 60k to 180k

The run resumes our own verified 60,000-step checkpoint and schedules 120,000 additional updates (15,360,000 example presentations). It keeps the mixed sampler, batch 128 and BF16 training. Source commit: ca26869455692cb570b94cb4a633be7701221f9e. Weights and exact predictions are saved to Drive every 20,000 updates before export and bot evaluation.

Step	Validation token loss	Public correct / 960	Public score
80,000	0.5250	414	43.13%
100,000	0.4909	455	47.40%
120,000	0.4712	472	49.17%
140,000	0.4528	487	50.73%
160,000	0.4419	516	53.75%
180,000	0.4402	517	53.85%

The saved completion record reports return code 0 and 6 checkpoints. All six were scored. The last 20,000 updates added one public correct answer (516 to 517). Training was stopped at its planned endpoint, GPU release was verified, and subsequent file work ran on CPU.

Each export is checked against its Drive manifest, source commit, checkpoint hash, official question-order hash and exact decoded selected token IDs. Bot receipts are written and read back on Drive. The current-generation measurements use Blackwell and do not establish T4 compliance.

Generation and speed

Measured optimization experiments

The course materials motivated length grouping, mixed precision, gradient accumulation, clipping and caching. The model already uses cached autoregressive generation. Training disables that cache. Loss accumulation stays on the GPU until logging. These choices were measured rather than assumed to make the full workload faster.

Blackwell inference, same weights	Median seconds	Changed outputs
Before, FP32	11.350	0
After, FP32	11.240	0
After, FP16	13.027	54
After, BF16	12.249	313

Two repetitions used reversed variant order on the second pass. The approximately 1% FP32 difference has overlapping timing ranges; it is not convincing evidence of a speedup. Lower inference precision was slower and changed programs. FP32 remains the default. An 80-update continuation benchmark also found no clear speed or validation-loss improvement; it does not establish full retraining quality.

T4 requirement

The historical 39.09-second T4 measurement belongs to an older run. Blackwell timings cannot establish the T4 requirement for the current candidate. A bounded final-checkpoint T4 experiment is reported separately below; only its measured configurations can establish compliance.

Exact model output and submission

Generation sorts questions by token length and restores their original positions. Historical runs followed greedy decoding with four-beam generation for syntactically invalid outputs. As of 16 October, strict greedy is selected: one candidate, no sampling, no syntax fallback, and no execution voting. No candidate is rewritten. Selected token IDs are saved, then decoded again to check equality with every output string. A sidecar records source commit, checkpoint hash, question-order hash, output hash, device and generation settings. JSONL conversion only wraps each exact string as a code field.

The historical submission included one handwritten identifier repair, which does not meet the user's stricter model-only condition. That repair has been removed from all future inference. The old score is retained as historical evidence, not a score for a newly verified model-only candidate.

The user confirmed that the required input is the official test.parquet question column in its original 1,919-row order. Generation preserves that order. The separate pass1_questions.jsonl is no longer a prerequisite.

Agent use and reproducibility

What the agent did

Codex read the assignment and course weeks, wrote data access, training, inference and diagnostic code, controlled Colab through MCP, inspected GPU state, compared measurements, prepared Telegram file packaging and wrote this report. It identified incomplete validation exclusion, stale T4 claims and handwritten postprocessing. Those findings changed the data exclusion rules, provenance records and speed checks.

Codex did not provide solution programs as training targets or replace model answers with its own solutions. No external model was used as a teacher. The submitted programs must come from the trained Transformer. Bot examples are not used to tune individual answers. The humanizer skill is used only to make report prose direct; it does not conceal agent involvement.

What remains open

The early pilot weights were lost when temporary runtime storage was released before export succeeded; their measurements remain saved. The revised workflow passed CPU save and restore checks and retained all ten initial, eight first-continuation and six longer-continuation checkpoints, exact predictions and bot receipts in Drive. The training GPU was released. A child-process release request failed at the notebook disconnect step; the new inference notebook executes its final release from the notebook kernel and has a hard process timeout.

Longer training exceeded the 33% public baseline, but the first-place objective remains unmet. The best candidate and all intermediate checkpoints have actual bot scores. Changed decoding outputs require a new quality comparison; speed alone does not establish correctness. A public first-place result would not establish private first place. No private result is claimed.

Reproduction files

Repository: OsVox/hse-nlp, branch 2026, homeworks/hw2. The directory contains train.py, data.py, numeric_eval.py, inference.py, tokenizer files, saved model weights, inference and training notebooks, experiment records and submission receipts. quality_experiments.md specifies the controlled pilot before its run. The raw logs and run configurations accompany the quality results.

Use a fresh output directory for each experiment. Run training with the recorded settings and keep latest plus best checkpoints. Run numeric evaluation on reserved training rows only. Use hw2_inference_colab.ipynb on a T4 to generate and package the test outputs. Verify every checkpoint in Drive before releasing compute; export predictions and record bot receipts from Drive.

Sources

Course assignment; allowed dataset and schema; course week 2 (encoder-decoder attention and decoding), week 3 (training methods), week 4 (attention and caching). Local measurements: metrics.json, gpu_benchmark/*.json, quality_results.json when present, and submissions/2026-10-15-gpu-check.json.

Alternatives to more updates

Held-out decoding and checkpoint averaging

Both experiments used reserved training questions, with no public answers or bot failure examples used for selection. The numeric runner executes supplied references and exact model outputs with time and resource limits; 245 of 256 references were usable. Agreement is a diagnostic, not the course bot score.

Experiment	Numeric agreement	Generation seconds, CPU	Decision
120k greedy	120/245	10.512	Retain
120k four beams	125/245	57.190	5.44 times slower; not adopted
140k unchanged	122/245	10.160	Retain
Average of 120k + 140k	124/245	10.313	Mixed evidence; not adopted

Averaging used identical architecture, tokenizer and training-run settings. Parent weights were verified and averaged in CPU float64, then cast back and reopened. It adds no training targets or extra inference model. Validation token loss worsened from 0.452825 to 0.459383, and execution failures rose from 16 to 22. The two-answer diagnostic gain does not justify replacing the parent.

Scaling-law estimate

The existing train-only probe contains 15,358 rows, sampled about equally across the 60 shards. With our tokenizer and length limits, mean question length is 46.91 tokens and mean code length is 73.83. Extrapolation gives approximately 555 million source tokens and 874 million target tokens per corpus. The last shard is smaller, so these are rough, non-deduplicated estimates. About 0.15% of sampled questions and 1.94% of sampled programs exceed the length limits.

Chinchilla suggests about 20 training tokens per parameter as a rough compute-efficient guide. Dividing target tokens by 20 gives 44 million parameters; counting source plus target gives 71 million. These are two accounting proxies, not a confidence interval or a fitted optimum for encoder-decoder code generation. Encoder and decoder capacity have different scaling behavior, and lower token loss need not imply higher task accuracy.

Proposed first larger model: width 512, feed-forward width 1536, six encoder layers and four decoder layers, totaling 42,481,152 parameters. Keeping four decoder layers limits repeated work during generation. At unchanged efficiency and time proportional to parameter count, one corpus-sized amount of data would take roughly 3.8-4.4 hours on the measured training GPU. This is a planning estimate; a short throughput/quality pilot and actual T4 timing must precede long training.

Hoffmann et al., Training Compute-Optimal Large Language Models; Ghorbani et al., Scaling Laws for Neural Machine Translation. Reproduction: scaling_estimate.py and experiments/scaling-analysis/.

Final-checkpoint T4 inference

Protocol

Prepare and verify model weights, tokenizer, official 1,919 questions and source on CPU. Save a hash-verified bundle to Drive, export and verify its ready record, then allocate one T4. Compare FP32 batch sizes 64, 128 and 256, twice with reversed order. Keep greedy generation, cached decoder state, length sorting, syntax-only beam retry and input/output limits 128/192 unchanged. Save every exact output, token provenance, hash and measurement to Drive. The two-model run omits optional profiling to keep GPU use bounded.

Generation time includes tokenization, sorting, generation, retries, decoding and local CSV writing. Full-call time additionally covers test reading, model hashing, provenance writing and verification. Model load and a small warm-up are measured separately. Drive archival is outside the generation timer. The experiment has a 600-second soft budget and a bounded child-process timeout; the notebook requests GPU release in finally.

Model	Batch	Median, s	Slowest, s	Same outputs as 64
9.97M, 180k	64	28.885	29.122	True
9.97M, 180k	128	16.676	16.738	True
9.97M, 180k	256	10.586	10.791	True
42.48M, 6k	64	37.032	37.585	True
42.48M, 6k	128	23.146	23.502	True
42.48M, 6k	256	20.304	20.556	True

Device: Tesla T4; PyTorch 2.11.0+cu130; Transformers 4.57.6. Model loading took 0.289 seconds and warm-up took 0.954 seconds. Changed programs require separate quality evaluation; byte-identical outputs preserve the same predictions.

Batch 256 is selected for these immutable weights. Both models finish far below 180 seconds. Every run matches its already-scored JSONL exactly: 53.85% for the established model and 18.96% for the 6k larger pilot. Duplicate bot submissions were avoided. Later weights need a fresh timing and output check.

Full runs, per-call times, memory peaks, provenance and output hashes are retained in experiments/t4-comparison-v1 and its Drive archive. Reproduce with hw2_t4_comparison_colab.ipynb; the main inference notebook now selects batch 256 for the established checkpoint. The completed T4 runtime was manually released after its asynchronous release request left it connected.

Short architecture and rate pilots

Four fresh models, 2,000 updates each

The screen kept seed 42, batch 128, BF16 training, the same mixed data stream, tokenizer and 128/192 token limits. All runs used 200 warm-up updates and a short cosine schedule. Width, encoder depth and learning rate were varied in separate comparisons. One immutable checkpoint per trial was saved in Drive, exported with hashes and selected token IDs, verified locally, then scored once by the bot. Source: 1f1558515ce00d1be29f032bac843bcbfc5b6bf4.

Parameters / layers	Peak LR	Val. loss	Held-out / 245	Public / 960	Train + export, s
9.97M, 4/4	.001	1.5262	18	66 (6.88%)	106.67
35.66M, 4/4	.001	1.9631	12	31 (3.23%)	181.02
42.48M, 6/4	.001	2.0145	9	27 (2.81%)	193.07
42.48M, 6/4	.0003	1.6312	23	86 (8.96%)	190.10

Keep the 9.97M model at .001 and the 42.48M model at .0003. The small model has lower token loss and lower cost; the larger finalist has higher held-out execution agreement, confirmed by the bot. Lowering the rate within the 42M architecture improved all quality measures. The larger high-rate models do not justify continuation from this screen.

These are early-learning results from one seed, not final convergence estimates. Even the best pilot remains below the established 180k model at 53.85%. The bot reported public rank 10/15 on 16 October; earlier rank 9/15 was an earlier snapshot. No private score is available. The later T4 comparison covers the established model and the larger 6k finalist.

Time and next comparison

The driver took 718.51 seconds after data restoration; its launcher record spans 758.79 seconds including restoration and imports. Across 716 GPU samples, mean activity was 72.90%, median 97%, and peak memory 18,627 MiB. Activity is not achieved compute throughput; startup, I/O and execution evaluation contribute low samples. Batch-size speed still needs a matched benchmark.

Continue both finalists from 2k to 6k with their optimizer moments, batch 128 and respective learning rates. Both receive the same new deterministic sample stream, a restarted 4k cosine schedule and 200 warm-up updates. Save and score the 6k checkpoint for each. Check actual T4 inference before long training. The four-way GPU was released manually after the automatic notebook request left it connected; file work resumed on CPU.

Evidence: experiments/short-pilots-v1, its selection.md and export.json, four submission receipts, finalist_pilots.py and hw2_finalist_pilots_colab.ipynb. Full weights, optimizer states and exact predictions remain in Google Drive.

Finalists after 6,000 updates

Matched continuation

Both models resumed their own 2k weights and Adam moments. Batch 128, BF16, original targets, the same new mixed sample stream, a restarted 4k cosine schedule and 200 warm-up updates were held fixed. Peak rates remained .001 for 9.97M and .0003 for 42.48M. Source: 10db3cda45ba777e04e4195f003a6b1db5254b24.

Model	Val. loss	Held-out /245	Public /960	Added train + export, s
9.97M	0.9463	38	154 (16.04%)	183.73
42.48M	0.9782	41	182 (18.96%)	353.36

The larger model leads execution accuracy in both evaluations; the smaller model has lower next-token loss and lower cost. The public difference is 28 answers (2.92 percentage points), while the held-out difference is only three answers. One seed and early checkpoints cannot establish an optimal model size or statistical significance. Both remain below the established 180k model at 53.85%.

Measured cost

The added training loop took 160.2 seconds for 9.97M and 333.3 seconds for 42.48M. The wider timings in the table include startup, validation, checkpoint saves and all-test generation. At the larger model's measured rate, a corpus-sized amount of examples would take about 2.1 hours of training loop time at batch 128, before evaluation and archival. This improves on the older 3.8-4.4-hour parameter-linear estimate but remains a short-run extrapolation.

Across 583 GPU samples, median activity was 96%, mean activity 72.15%, and peak used memory 18,631 MiB. Activity includes I/O and numeric evaluation and does not measure achieved compute efficiency. A separate batch probe uses the actual encoded/prefetched training input path, equal examples, reversed rounds and timing-boundary synchronization.

Checkpoint and evaluation retention

Both immutable 6k checkpoints, optimizer states, predictions and validation records were saved to Drive. The evidence archive was created there before download. Local checks verified hashes, all 1,919 official question positions and exact decoding of selected token IDs. Each checkpoint was submitted once to the course bot. Receipts are retained separately from the pre-submission archive. No answer text was edited.

The G4 runtime was manually released after its asynchronous release request left it connected. Standard CPU was used to stage the established 180k model and the larger 6k finalist for actual T4 inference. A faster training batch must also pass a short equal-example quality check before being adopted for full training.

Evidence: experiments/finalist-pilots-v1, its selection.md and export.json, the two 6k submission receipts, experiments/t4-comparison-v1/ready.json and benchmark_production_batches.py.

Larger model: speed and quality

The 42.48M model used one prepared G4 session for a speed probe, short quality checks and a capped continuation. All eight probe passes used 32,768 original examples and 2,419,826 target tokens; the second round reversed the batch order. Probe weights were discarded.

Batch	Median examples/s	Peak allocated GiB
64	1794.6	8.18
128	1623.4	15.78
256	1433.7	30.97
512	1319.9	61.34

Batch 64 was 10.55% faster than 128. Both quality pilots resumed the same own 6k weights and optimizer state, with fixed added example counts, BF16 and peak rate .0003. The predefined gate required no fewer correct held-out executions and token loss within 1% of the control.

Batch	Added examples	Token loss	Held-out /245	Public /960
128	262,144	0.8854	45	183
64	262,144	0.8779	41	182

Batch 128 was retained. Lower token loss at batch 64 did not compensate for its lower execution accuracy. The chosen lineage continued to 2,560,000 total example presentations, equivalent to 20,000 updates at batch 128; this is not a full pass over the 11.84M-row corpus.

Continuation step	Token loss	Public correct /960
10,000	0.8731	135
15,000	0.6995	266
20,000	0.6167	320

The 10k public regression followed a learning-rate restart; causality was not established. The final held-out result was 75/245. The grouped driver took 27.05 minutes including probes, preparation, evaluation and Drive saves. Each checkpoint retained exact token provenance and a separate bot receipt; no program was rewritten.

Decoding opportunity

On the established 180k model, four-beam decoding improved held-out numeric agreement from 131/245 to 137/245 and reduced execution failures from 19 to 11. CPU generation took 10.61 versus 50.37 seconds. The subsequent full test export scored 544/960 (56.67%), versus 517/960 (53.85%) for greedy decoding. All 1,919 questions took 503.50 seconds on CPU at batch 64, FP32. T4 timing for this output remains unverified and is deferred. Every program matches its saved model token decode; no program was rewritten.

Source: 85532a78132c37ceb9cbf3f6cfff5c478d69258f. Evidence: experiments/grouped-42m-20k-v1, experiments/decoding-180k and experiments/beam4-quality-180k-v1 (source d1aec6e). Results were downloaded with the authenticated local Drive CLI, avoiding Colab allocation for export.

Full model and decoding checks

The 42.48M model resumes its own 20k weights and Adam moments. The original target was 100k updates; the user requested a shift away from scaling, so the run was intentionally stopped after saving 70k. The GPU session was released. Settings were batch 128, BF16, peak learning rate .00015, 500 warmup updates, cosine decay and the encoded/prefetched input pipeline. Halving the prior peak is a restart hypothesis, not a fitted optimum. The first 30k and 40k bot scores are review points.

Before allocating G4, CPU checks verified all dataset hashes, loaded the model and optimizer, checked finite optimizer moments and saved a verified Drive bundle. At startup the GPU showed 98% activity and 18,951 MiB used out of 97,887 MiB. This is an observed snapshot, not a utilization guarantee. Each 10k checkpoint saves weights, optimizer state, exact predictions and a separate bot receipt.

Step	Token loss	Public correct /960	Public %
20,000 (parent)	0.6167	320	33.33
30,000	0.5475	409	42.60
40,000	0.4904	471	49.06
50,000	0.4516	519	54.06

At 3.84M example presentations, the larger model scored 409/960, versus 273/960 for the smaller model at step 40k. Different learning-rate schedules and data streams prevent a causal size-only comparison. These are scored checkpoints, not a completed target run. These historical exports use greedy decoding with syntax-only beam retries. No test labels are used for training. Source: 69829cbe; evidence: experiments/full-42m-100k-v1.

Execution agreement without new training

Using the fixed 9.97M, 180k model, generate four deterministic beam candidates and execute their solve() functions in the restricted diagnostic runner. Select an unchanged program from a strict majority of all four numeric answers; otherwise retain the first executable candidate, breaking ties by model rank. References enter only after selection, for evaluation. No program is rewritten or added to training.

Selection	Rows 512:768 (247 usable)	Rows 768:1024 (246 usable)
Top beam	141	136
First executable	142	136
Strict majority	142	138

Strict majority passed the predefined positive-gain gate on both subsets: three extra correct answers across 493 usable references. This small exploratory gain is not statistical significance. First-executable selection failed the confirmation gate. Candidate and selected token decodes were checked after Drive export. The historical validation-copy caveat still applies to the 180k lineage.

Both tests ran on four CPU cores of the 48-core training host with CUDA hidden; GPU training continued. The full-test export scored 548/960 (57.08%), four more than beam-only decoding; CPU generation and selection took 279.43 seconds. T4 timing is unverified. Inspiration: Wang et al., Self-Consistency; our deterministic beam experiment differs from their sampling method. Sources: f047d7d (pilot), 124e46d (test export). Evidence: experiments/candidate-vote-180k-v1 and candidate-vote-test-180k-v1.

Selected policy: strict greedy decoding

The user selected greedy decoding on 16 October. All new evaluated exports use num_beams=1 and do_sample=False, with syntax retries and execution voting disabled. Historical results are preserved with their original settings. Training stopped after its durable 70k checkpoint; separate CPU export processes apply strict greedy to saved weights.

Step	Token loss	Public correct /960	Public %
50,000	0.4516	519	54.06
60,000	0.4255	540	56.25
70,000	0.4135	554	57.71

The 50k strict greedy output scored 519/960 (54.06%), matching the public score of its earlier retry-enabled output. Two of the 1,919 output strings differed; the comparison also changed device and batch size, so this is not a controlled attribution of the difference to retries alone. CPU generation took 99.67 seconds at batch 64. This is not T4 timing.

The default-decoder regression test supplies malformed Python and verifies that it is preserved with no extra generation calls. Full exports verify every program against saved token IDs, the official question-order hash, checkpoint hash and source commit. Each output is first saved to Drive, exported with the authenticated CLI, checked again and submitted once. Receipts are read back from Drive byte-for-byte.

Source: 8dcb74d2164a30c7b3169dcf7adf76114139761f. Evidence: experiments/decoding-policy.json, experiments/strict-greedy-42m-v1 and subsequent strict-greedy-42m checkpoint directories. The main inference notebook follows this policy; the earlier voting notebook is retained only for historical reproduction. First place remains unmet.

Checkpoint averaging test

Arithmetic averaging combines our own 60k/70k or 50k/60k/70k weights into one model of unchanged size. It adds zero training updates and still generates one greedy candidate. All runs use CPU, FP32, batch 64 and two threads. The original Transformer paper also used checkpoint averaging, although its decoding settings differed.

Model	Pilot correct /247	Execution failures
baseline	142	17
avg60_70	152	15
avg50_60_70	141	14

Model	Confirmation correct /246	Execution failures
baseline	135	10
avg60_70	136	13

Predefined gate: strictly more numeric agreements and no more execution failures than 70k on both held-out samples. Selected candidate: avg60_70; confirmed: False. These are small exploratory samples, not a significance test or public pass@1. Public test answers never choose the average.

Both candidates and their parent hashes are retained on Drive. Original held-out training references are used only for evaluation, with no generated training targets or handwritten repairs. Source: 861ae9795086a67af530f8eb2fc873423f466fbd. Evidence: experiments/checkpoint-average-70k-v1. Reference: Vaswani et al., Attention Is All You Need, section 6.1.

Bot result for baseline: 554/960 (57.71%). Exact file: strict-greedy-42m-070000.jsonl. T4 timing remains unverified.

Short matched loss-weight test

Two independent 2,000-update continuations start from the same own 70k weights and optimizer state. Both use batch 128, seed 42, identical sampled data, BF16, peak learning rate .00003, 100 warmup steps and the same cosine schedule. Control uses the original loss. Numeric2 gives twice the weight to 305 of 8,192 tokenizer pieces matching numbers or arithmetic operators. Original programs are unchanged; matching pieces in comments and strings also receive weight. This is a heuristic, not a syntax-aware objective.

Arm	Pilot agreements	Confirmation agreements	Public correct /960
control	141/245	284/493	562
numeric2	138/245	289/493	573

The predefined held-out gate requires more numeric agreements and no extra execution failures on both subsets. Confirmed: False. This is a short exploratory comparison on previously used validation questions; it is not statistical significance. Training losses across arms are not directly comparable. Uniform validation loss and executed-answer diagnostics remain comparable.

Five focused tests checked normalized weighting, ignored padding, piece selection and continuation scheduling. CPU checks verified all dataset files, model weights and optimizer moments before G4 allocation. Both checkpoints and exact greedy outputs were saved to Drive before local export, token verification and bot submission. The control scored 562/960 (58.54%) and numeric2 scored 573/960 (59.69%), an 11-answer public gain at equal training budget. Numeric2 was retained at that stage for reproducing the highest public score; it did not pass the held-out gate for extending training. Total experiment time was 453 seconds, median GPU activity 97%. Colab showed no active sessions after completion. No longer training run started. Source: 8755a8f4867683d940e9b34fa48365e3cf4f8c21; evidence: experiments/numeric-loss-2k-v1. T4 timing remains deferred.

Broad short experiment screen

Eight matched continuations start from the same 42.48M model at step 70,000, including its optimizer state. Each gets only 1,000 updates at batch 128. Seven candidates change one factor relative to control: numeric token weighting, label smoothing, equal question weight, dropout 0 or .05, doubled peak learning rate, or excluding targets longer than 192 tokens. Targets are never rewritten. Filtering refills complete batches but necessarily changes which examples are seen.

Arm	Held-out /738	Failures	Public /960
control	416	40	562
numeric2	417	42	562
smooth005	425	44	555
equal_question	416	38	562
dropout0	414	38	564
dropout005	413	40	559
lr6e5	419	40	559
complete_targets	423	45	555

All outputs use strict greedy decoding. The shortlist requires positive total held-out gain, no negative gain on either validation subset, and no increase in total execution failures. At most three candidates proceed to a second-seed test against control. Combinations need their own short comparison before a long run; individual gains are not assumed to add together.

Shortlist: none. These previously reused validation questions provide exploratory evidence, not an untouched estimate of generalization. Public bot scores are recorded separately; individual public failure examples are not used to design targets or repairs.

Seventeen focused checks passed before CPU verification and GPU allocation. Checkpoints, exact greedy predictions, source identity and GPU telemetry are saved to Drive before export. Source: 5de771fc79b3b1e84926aedd8292560e56d9fecd. Evidence: experiments/broad-screen-1k-v1. Label smoothing and dropout motivation: Vaswani et al., section 5.4. Their usefulness here is determined by the measured trials.

Driver duration: 16.64 minutes, including data restore, training, evaluation and saves. Colab allocation and Drive sign-in occur before this timer. No long run is launched automatically.

Training distribution audit

Before another GPU experiment, scan all 60 original training shards on CPU and exclude every exact copy of the 1,024 held-out question texts. Count exact questions using BLAKE2b-128 hashes and draw a fixed sample of 2,048 original pairs per shard for tokenizer statistics. No training targets are created or rewritten.

Measurement	Observed
Eligible original rows	11,836,444
Distinct question hashes	10,920,597
Excess duplicate rows	7.74%
Rows belonging to repeated questions	13.48%
Sample size	122,880
Questions exceeding 128 tokens	0.173%
Programs exceeding 192 tokens	1.946%
Median / mean program tokens	65 / 74.48

Exact duplication and truncation affect a minority of rows. This does not prove filtering cannot help, but it does not establish a large data-cleaning opportunity. The audit is descriptive: it does not assess semantic correctness, normalized paraphrases, or the effect of changing the training distribution.

A separate code inspection found that grouped batches always run in ascending input-length order within each window. An optional deterministic shuffle now changes only batch order, preserving every batch tensor and its padding cost. Thirteen focused checks passed, including unchanged default behavior and exact shuffled batch membership. This option is prepared for a short two-seed comparison; no score gain or completed training run is claimed.

Audit computation: 59.14 seconds; CPU process, input restore and setup: 85.97 seconds. No GPU was allocated. Colab confirmed no active sessions after automatic release. The contemporaneous public leaderboard remains 8/15: 59.69% versus 78.33% for first place. Evidence: experiments/train-diversity-audit-v1; audit source: 6add585. The first-place objective remains unmet.

Grouped batch order: two-seed test

Four continuations start from identical own 70k weights and Adam moments. Within each seed, the control processes length-grouped batches in ascending length order; the treatment shuffles these same completed batches within each 4,096-example window. Both retain identical batch membership, padding cost and original targets. Seeds 42 and 1042 change the data/dropout stream. Each arm gets 1,024 updates, batch 128, or 131,072 example presentations; the budget ends on a complete grouping window.

Seed / order	Pilot /245	Confirm /493	Failures	Public /960
control_s42	137	281	41	561
shuffled_s42	136	285	45	550
control_s1042	138	284	49	560
shuffled_s1042	135	286	48	549

The predefined gate requires positive total held-out gain in each seed, no negative gain in either subset for either seed, and no increase in execution failures pooled across seeds. Confirmed: False. Reused questions across seeds are not independent test observations, and all validation remains exploratory.

The shuffled order gained 3 held-out correct answers in seed 42 but lost 1 in seed 1042, with 3 additional pooled execution failures. Public accuracy fell by 11 correct answers in each pair. No long extension is justified. The retained best remains 573/960 (59.69%), rank 8/15.

All arms use unchanged architecture, original token loss, peak learning rate .00003, 50 warmup updates and strict greedy decoding. Fifteen focused tests passed, including exact preservation of shuffled batch tensors, unchanged default behavior, checkpoint boundaries and paired selection. Every checkpoint and test output is saved to Drive before export and verification. Source: 862fa6f491c6c1bd4c7977d688aa37deaf2c54db; evidence: experiments/batch-order-1k-v1.

Driver duration: 8.60 minutes, excluding allocation and authorization before the driver. Median sampled GPU utilization was 97%; mean was 63.79%, including restoration, CPU scoring and saving. Colab confirmed no active sessions after automatic release. No long extension was started.

Individual digits: short adaptation test

The treatment adds individual digit boundaries before BPE merges while retaining the original learned vocabulary, token IDs, model architecture and target text. The control and treatment continue from identical own 70k weights and Adam moments for 1,024 updates each, with batch 128, peak learning rate .00003 and 50 warmup updates. Both use strict greedy generation. The same original example pool is consumed at the complete-window boundary; longer sequences change grouping and padding.

CPU audit: 122,875 original pairs sampled across all 60 shards, excluding held-out question copies. Both tokenizers recover every original text exactly; save/reload and unchanged vocabulary IDs passed. Question mean length rises from 46.97 to 51.85 tokens; program mean from 74.52 to 80.94. At the unchanged 128/192 limits, question truncations rise from 218 to 446 and program truncations from 2,375 to 3,005.

Tokenizer	Pilot /245	Confirm /493	Failures	Public /960
control	137	281	41	561
digits	124	239	47	511

The predefined gate requires improvement on both held-out subsets and no increase in total execution failures. Shortlisted for a second-seed test: none. No long extension is automatic. This tests adaptation of an existing BPE model, not a model trained from scratch with digit boundaries. Repeated validation is exploratory.

The treatment lost 55 held-out agreements and added 6 execution failures. Public accuracy fell by 50/960 (5.21 percentage points). Reject this adaptation; the best remains 573/960 (59.69%), rank 8/15. The original control reproduces previous correct counts but 11 of 1,919 program strings differ, so bitwise repeatability is not established.

Motivation: Singh and Strouse, Tokenization counts (2024), https://arxiv.org/abs/2402.14903. That paper concerns arithmetic tokenization and does not establish a benefit for this Python-generation task. No paper examples or additional targets enter training.

Eighteen focused checks passed before launch. Every checkpoint and its original test output was saved to Drive before export and exact verification. Training source: f9676760c37607102647c7d0b792810f81f1e8f1. Evidence: experiments/digit-screen-1k-v1.

GPU driver duration: 4.90 minutes, excluding allocation and authorization. CPU preparation took 149.47 seconds, including the 65.65-second tokenizer audit. Colab confirmed no active sessions after automatic release; T4 timing remains unverified.

Focal token loss: short matched screen

Three continuations start from the same own 70k model and Adam state. Each receives 1,024 updates, batch 128, peak learning rate .00003, 50 warmup updates and unchanged original data, vocabulary and programs. The control uses cross-entropy. Focal1 and focal2 multiply each original target-token loss by (1-p) raised to gamma=1 or 2, where p is its predicted probability. The gradient includes this factor; alpha=1. The denominator remains the original valid-token weight. Greedy decoding is unchanged.

Loss	Pilot /245	Confirm /493	Failures	Public /960
control	137	281	41	561
focal1	139	283	40	559
focal2	138	278	42	561

The predefined gate requires improvement on both held-out subsets without more total execution failures. Shortlisted for second-seed confirmation: focal1. No automatic combination or long run. Focal modulation changes gradient magnitude as well as emphasis, so identical nominal learning rates do not imply identical effective steps. Reused validation remains exploratory.

Evaluation now saves the exact generated programs and original references before scoring. Separate CPU-only scorers run while later GPU work proceeds. Payload hashes and row order are verified before scoring; the scoring branch imports no torch and does not load model weights. Process failures propagate to the driver, and pending processes are cleaned up. Twenty focused checks passed, including focal values/gradients, padding, score preservation and failure handling.

The verified dataset and parent archive from the earlier CPU preflight are reused; new source and launch files are uploaded to Drive and read back before GPU allocation. Archive hashes are rechecked on restore. Checkpoints and original test outputs are saved to Drive before export and bot scoring. Source: 86c3bfb603eeb2f20c7e3b4cf7dd3f246b084d93; evidence: experiments/focal-screen-1k-v1.

Motivation: Lin et al., Focal Loss for Dense Object Detection, ICCV 2017. Its object-detection results do not establish a benefit for this sequence-generation task. No external examples or additional training targets are used.

Focal1 gains four held-out agreements with one fewer execution failure, but its public score is two answers below control. Focal2 fails the held-out gate. No long run is justified; focal1 is only a candidate for short second-seed confirmation. The best remains 573/960 (59.69%), rank 8/15.

Driver duration: 6.35 minutes, excluding allocation and Drive authorization. Median GPU activity was 97%, mean 65.68%. Only 3 of 34 samples during CPU scoring show nonzero GPU activity, none at 90%: overlap was mostly with setup, and speedup is not established. Colab confirmed no active sessions. All three bot receipts are saved and verified in Drive. T4 timing remains unverified.

Focal loss: second-seed confirmation

The gamma-one candidate alone is repeated against control with seed 1042. Both new arms start from the original own 70k parent and Adam state, not from the first-seed results. Each receives 1,024 updates with the same batch size, learning-rate schedule, data pipeline and strict greedy decoding. Seed 42 results are reused without retraining.

Seed / loss	Pilot /245	Confirm /493	Failures	Public /960
42 / control	137	281	41	561
42 / focal1	139	283	40	559
1042 / control	138	284	47	560
1042 / focal1	138	286	46	563

The predefined confirmation gate requires positive gains on both held-out subsets in each seed, with no increase in total execution failures in either seed. Confirmed: False. Public scores do not enter this gate. The same held-out questions are reused across seeds, so this is a robustness check rather than an independent validation set.

Twenty-one focused checks passed, including rejection of zero subset gains, extra execution failures and a failing first seed. The source archive includes the exact first-seed selection evidence and records its hash. Unchanged data and parent archives reuse the earlier CPU preflight; source and launcher are uploaded and read back before GPU allocation. Each new checkpoint and its predictions are saved to Drive before export and bot evaluation.

Source: bbf58a9376348573d28a72b798113fa0d6bab6a9. Evidence: experiments/focal-confirm-1k-v1. No additional training examples, target rewrites or handwritten output repairs are used. No long run starts automatically. T4 timing remains deferred.

The second seed gains two held-out agreements with one fewer failure, but the pilot subset ties control, so the predefined confirmation gate fails. Public gains are -2 and +3 across seeds. No long extension is justified. Best remains numeric2 at 573/960 (59.69%), rank 8/15. Next proposed check: a CPU audit of consistency among original duplicate-question programs, without creating targets.

The driver took 269.42 seconds (4.49 minutes), excluding allocation and authorization. GPU median activity was 97%, mean 64.09%, peak sampled memory 21,309 MiB. Both checkpoints and bot receipts are saved in Drive. Colab Manage Sessions confirmed no active sessions after automatic release.

Python tokenizer: CPU comparison

At the user's suggestion, fit four new tokenizers on 61,440 original question/program pairs sampled across all 60 shards. Evaluate on 15,304 separate exact question texts; remove all held-out question copies from fitting and this audit. The existing 8,192-token byte BPE is retained as a reference. It was previously trained on shard 0, so its compression comparison is not an untouched generalization evaluation.

Tokenizer	Question mean	Code mean	Code >192	Code delta
original8192	46.86	74.11	278	+0.00%
balanced8192	46.50	73.30	250	-1.08%
code8192	46.98	73.09	250	-1.36%
python8192	83.65	86.21	556	+16.34%
code16384	45.44	71.76	219	-3.17%

Balanced8192 sees each original question and code once. Code8192 repeats the unchanged original code three times per question during tokenizer fitting. Python8192 adds explicit lexical-style boundaries for identifiers, numeric strings, operators, whitespace and indentation. Code16384 doubles the vocabulary with the same code emphasis. No pretrained vocabulary, new examples or rewritten training targets are used.

Every candidate is serialized, reloaded and checked for exact recovery of questions and code, including significant whitespace. Unknown tokens, number fragmentation and truncation counts are recorded. Shorter sequences do not prove better executed-answer accuracy. A different vocabulary cannot silently replace token meanings in the trained checkpoint; any deployment requires a matched training experiment.

Five focused local checks passed, including saved tokenizer round trips with tabs, CRLF and Unicode, disjoint question sampling, unchanged original Parquet files and separation of numeric conflict from unsupported execution. Motivation: Hugging Face CodeParrot methodology, https://github.com/huggingface/blog/blob/main/codeparrot.md. That work motivates a code-specific tokenizer; it does not prove a gain on this task.

Source: 52cccd19da35f70d023cf63f2aeb0d161524de18. Evidence: experiments/cpu-data-tokenizer-audit-v1/tokenizers. All work uses CPU; artifacts are saved to Drive before export.

The code-weighted 8k tokenizer shortens programs by 1.36% versus the current tokenizer, but only 0.28% versus the fresh balanced control. The 16k version shortens code by 3.17% and reduces over-limit programs from 278 to 219 of 15,304; at width512 its shared token table adds 4,194,304 parameters. The tested explicit Python boundaries lengthen programs by 16.34% and questions by 78.53%, so reject this particular boundary rule. All variants round-trip exactly with zero unknown tokens. Retain code8192 and code16384 as a matched vocabulary-size pair for a short future training test; compression does not establish an accuracy or wall-time gain. No tokenizer has replaced the best checkpoint.

Original target consistency: CPU audit

Sample exact training-question groups using a fixed keyed hash, independently of public/test examples. Scan all original shards and exclude every held-out question copy. Among repeated questions with different original programs, test at most 512 groups and eight distinct programs per group. Execute only original programs with the existing restricted diagnostic runner; no values are converted into training targets.

Measurement	Count
Eligible original rows	11,836,444
Sampled exact question groups	10510
Repeated question groups	701
Repeated groups with identical program	198
Repeated groups with different programs	503
Tested groups / programs	503 / 1126
Groups with program cap	5
all_executable_agree	284
numeric_conflict	177
insufficient_numeric	38
partial_agreement	4

Numeric conflict means at least two executable original programs disagree within the existing 1e-6 tolerance. It does not identify which program is correct. Agreement also does not prove correctness. Unsupported or failed execution remains unresolved rather than an automatic reason to discard a target. Groups are sampled by question, not weighted by frequency; the result cannot be read as the fraction of all training rows that are wrong.

The audit saves question/program hashes, original shard/row positions, statuses and aggregate comparisons. It leaves original files unchanged and emits no training target file. Evidence: experiments/cpu-data-tokenizer-audit-v1/consistency.

Among 503 sampled repeated questions with distinct original programs, 177 (35.19%) have at least two conflicting numeric results; 284 fully agree, 38 have insufficient executable results, and 4 have partial agreement. Five groups hit the eight-program cap. This supports investigating a cheap original-data filtering experiment, but gives no authority to choose a correct answer or create targets. No filtering has been applied.

The complete CPU driver took 201.29 seconds (3.35 minutes), excluding allocation and Drive connection. No GPU was allocated. It exited successfully and Colab Manage Sessions confirmed no active sessions. Tokenizer artifacts, sampling provenance and audit results were saved to Drive before export and hash verification.

Comments and empty lines: CPU preflight

At the user's suggestion, test removing non-executing text from existing training targets. The cleaner uses Python tokenization to distinguish comments from hash characters inside strings. Empty lines inside strings and explicit line continuations are preserved. Every accepted change must compile and have the same complete parsed Python structure, excluding source positions. Invalid or structurally changed programs fall back to the original.

Measurement	Result
Sampled training programs	4096
Programs with comments	503
Changed / rejected	2923 / 0
Mean target tokens: original / cleaned	65.80 / 62.77
Target token change	-4.61%
Targets exceeding 192: original / cleaned	23 / 10
Executable pairs / identical numeric results	249 / 249
Numeric / status mismatches	0 / 0
Unsupported execution pairs	7

The fixed sample comes from training shard 0 and excludes every exact copy of held-out question texts. This is a single-shard compression diagnostic, not evidence of improved model accuracy. Docstrings, pass statements, imports, assignments, variable names and calculations remain intact. Source or line-number introspection is outside the equivalence claim.

The training experiment replaces each original target once; it creates no additional examples, alternate solutions or numerical-answer targets. Generated submissions remain exact greedy model output, with no cleanup after generation. Nine focused cleanup tests cover quoted hash characters, multiline strings, docstrings, Unicode, line continuation, invalid syntax, repeatability and preservation of the input example stream.

Representation: four short scratch trials

Compare original-tokenizer control, code-focused 8k vocabulary, code-focused 16k vocabulary, and comment/empty-line cleanup with the original tokenizer. Each receives 2,048 updates, batch 128, seed 42, learning rate 0.0003, and the same 262,134 original-example presentations (one batch has 118 rows). All models initialize at vocabulary 16,384; the 8k arms then shrink their tied embeddings. Their initial Transformer body hashes must match exactly. The dropout random stream is reset after shrinking.

Arm	Pilot /245	Confirm /493	Failures	Public /960
control	17	28	243	70
code8192	20	36	190	86
code16384	19	36	220	82
comments	17	34	217	65

The 8k models have 42,481,152 parameters; the 16k model has 46,675,456. Source stream and grouping-window budgets match, but changed lengths can change sorting inside a window. Token-level losses across vocabularies or target representations are not directly comparable. Evaluation uses original held-out programs and strict greedy decoding.

Predefined gate: improve both held-out subsets and add no total execution failures versus control. Candidates for a second-seed check: code8192, code16384. This early screen does not establish eventual performance after full training. Public results are recorded separately and do not enter the selection gate.

Source: 558c3fd580c18d8f7a1ca4a68a8126138fd3e062. Thirty-two checks passed before allocation. All checkpoints and predictions are saved to Drive before local export and bot submission. A hard runtime bound and automatic release are installed. No long run is started automatically.

Code8192 gains 11 held-out agreements and removes 53 execution failures; code16384 gains 10 and removes 23. Both improve each held-out subset and qualify for short second-seed checks. Comments gains six agreements but ties one subset, missing the strict gate; its public score also falls from 70 to 65. Do not combine cleanup or extend it into a long run on this evidence. Prefer the cheaper 8k tokenizer for confirmation first. Public scores are 70, 86, 82 and 65; all are early scratch models, not replacements for the retained 573/960 best checkpoint (rank 8/15). No full training starts.

The experiment driver took 858.86 seconds (14.31 minutes), excluding allocation, Drive reconnect and dependency setup. Colab showed an allocated GPU period of roughly 13:26–13:43 (minute precision). GPU median activity was 97%, mean 73.38%, peak sampled memory 22,739 MiB. The runtime was released automatically; Manage Sessions showed no active sessions. All four checkpoints and receipts are stored in Drive. T4 timing is unverified. At logged step 2,000, cleanup took 190.8 seconds versus 167.2 for control, so this implementation did not speed training. These training-loop times include CPU input preparation and exclude checkpoint export.

Code-focused 8k: second-seed check

Repeat the cheaper code-focused tokenizer against the original 8k control. Both new models start from random weights with seed 1042, receive 2,048 updates and use the same data, architecture, batch size, learning-rate schedule and strict greedy decoding. The tokenizer files are unchanged from the CPU fit and verified byte-for-byte. The initial Transformer bodies are identical within each seed; no target cleanup is applied.

Seed / tokenizer	Pilot /245	Confirm /493	Failures	Public /960
42 / control	17	28	243	70
42 / code8192	20	36	190	86
1042 / control	18	39	197	78
1042 / code8192	23	37	205	73

The confirmation gate requires strictly positive gains on both held-out subsets in each seed, with no increase in total execution failures in either seed. Confirmed: False. Both seeds reuse the same held-out questions, so this checks seed robustness rather than independent validation or statistical significance. Public feedback is recorded separately and never converted into training targets.

The first-seed evidence is copied into the source archive and hash-checked before the new run. Eighteen focused checks passed before allocation. Existing verified dataset, tokenizer and model-shape preflight evidence is reused without allocating a CPU Colab runtime. Source, launcher and manifest are saved to Drive and downloaded back before GPU allocation. Each completed checkpoint and exact model output is saved to Drive before bot evaluation.

Source: e463ac8cb58f6bb52608f6d3e067e3c1739020e5. Evidence: experiments/tokenizer-confirm-2k-v1. A 900-second driver bound and automatic runtime release are installed. A positive result permits considering a bounded extension, not an automatic full training run.

The code-focused 8k tokenizer does not pass the predefined second-seed gate. It gains five pilot agreements but loses two confirmation agreements, with eight more total execution failures. Public score is 73/960 versus control 78/960; seed 42 had 86 versus 70. This small, mixed result does not justify a long run or combination with cleanup. Retain the 573/960 (59.69%) best checkpoint; latest bot rank is 8/15. No full training starts.

The two-arm driver took 431.28 seconds (7.19 minutes), excluding allocation, Drive and setup. GPU median activity was 97%, mean 73.99%, peak sampled memory 19,147 MiB. The launcher exited successfully and released the runtime automatically; Manage Sessions showed no active sessions. Both checkpoints, exact greedy outputs and bot receipts are stored in Drive. T4 timing is unverified.

Original-data sampling: CPU preflight

Augmented question-program pairs are not used: the assignment prohibits additional training examples or targets. This experiment instead prepares inverse-frequency sampling of unchanged original rows. A question appearing f times is accepted with probability f to the power minus alpha. Alpha zero keeps the original sampler; 0.5 mildly reduces repetition; 1 applies full inverse-frequency weighting. All original program variants remain eligible. No answer is selected as correct and no program is rewritten.

The CPU index covers 11,836,444 eligible rows and 10,920,597 exact question strings across all 60 training shards. Every copy of a held-out question is excluded. One uint32 count is stored per original row; source and index checksums are verified before training. The existing mixed sampler draws equal candidates per shard, so this does not promise perfectly uniform question probabilities.

Sampling power	Accepted rows	Unique questions	Expected repeat share
0	245,750	244,692	13.44%
0.5	233,833	233,649	9.03%
1	226,787	226,719	6.22%

The table measures one fixed candidate pool, not model accuracy or equal-budget training. Weighted pools are smaller; a future fixed-update trial draws further original rows to fill the same number of training batches. Candidate selection uses no public test information. Prior numeric disagreements do not identify the correct reference, so they are not used to delete or replace targets.

Five focused tests cover exact cross-shard counts, held-out exclusion, checksum rejection, deterministic probabilities, original target preservation and unchanged zero-power behavior. Source: 1abd5e36874b45a5f9c28e8c4d06b3a0da617074. No GPU is allocated for this audit.

Power	Original rows	Unique questions	CPU loading seconds
0	131,072	130,717	21.42
0.5	131,072	131,007	24.16
1	131,072	131,047	14.79

This timing covers data loading and sampling only, without tokenization or model training. It is not GPU throughput. Unique counts in a short stream need not increase: repeatedly occurring questions can fall outside the selected prefix by chance.

The index is ready for a bounded matched sampling trial, not a long run. Full inverse-frequency sampling reduces expected exposure to corpus-repeated questions from 13.44% to 6.22% in the audited candidate pool. At equal 131,072-row budget, within-stream repeated copies fall from 355 to 25; this adds only 330 distinct questions, so a large accuracy gain should not be assumed. Targets remain unchanged and no conflicting reference is chosen as truth. The next preregistered comparison uses the best 72k checkpoint, numeric-token weight 2 in every arm and 1,024 updates per arm after CPU parent/resume preflight. There is no new model or bot score from this audit.

The CPU driver completed in 141.71 seconds (2.36 minutes), excluding allocation, Drive mount and dependency setup; index construction took 48.89 seconds. Sampling 131,072 rows took 21.42, 24.16 and 14.79 seconds for powers 0, 0.5 and 1 in that order. These are single, ordered CPU data-loading measurements affected by caching, not evidence of training speedup. All 60 exported count arrays passed their checksums. The runtime released automatically and Manage Sessions showed no active sessions. No GPU was allocated.

Inverse-frequency sampling: short training test

Three matched 1,024-update continuations from the verified 72k numeric-weight-2 checkpoint. The control preserves original mixed sampling. Inverse_sqrt and inverse accept an original row with probability 1/sqrt(f) and 1/f, where f is its exact question frequency across the training split. No input or target text is changed, and no reference is selected as correct. All arms retain numeric-token weight 2, the same optimizer state, batch size 128, learning rate 3e-5, 50-step warmup and strict greedy decoding.

Sampling	Pilot /245	Confirm /493	Failures	Public /960
control	139	286	40	575
inverse_sqrt	136	282	44	575
inverse	134	282	42	558

A candidate must improve both held-out subsets and add no total execution failures relative to control. Shortlist: none. A survivor needs a second-seed check before a longer run. Public scores are recorded separately and do not enter this selection rule. Repeated use of these validation questions and selection of the parent by public score make these results exploratory.

Eleven focused tests passed before allocation. CPU preflight verifies the parent weights, optimizer shapes and finite moments, resume step, a finite forward pass, the 60-shard frequency index and full batches at the actual continuation seed. Every model checkpoint and exact generated output is saved to Drive before export and bot evaluation. Training source: 0a3e2bfac3da70a5750efcf7233aa7506a6350e0. CPU data/model preflight used unchanged source 1d7ce0c3ec3ceae7d65e8d5153d308d5a5f69b8f. The first GPU attempt ended before training and left no checkpoint; Drive disconnected and its log was empty. Compute release was verified. The repaired launcher keeps its driver log locally and always requests compute release even if Drive export fails.

Reject both inverse-frequency treatments for longer training. Relative to control, inverse_sqrt loses 7 held-out agreements and adds 4 execution failures; inverse loses 9 and adds 2. Public results are 575/960 control, 575/960 inverse_sqrt and 558/960 inverse. Retain the simpler control for reproducing our new best 59.90%; the two-answer gain over its 72k parent is exploratory and does not establish a benefit from longer training. No long run or second seed is justified by this screen.

The successful three-arm driver took 367.16 seconds (6.12 minutes), with median GPU utilization 97%, mean 67.97% and peak memory 25,009 MiB. This excludes allocation/OAuth before the driver; the first failed attempt added approximately three minutes. CPU preflight took 122.83 seconds. Colab showed no active sessions after completion. T4 timing remains deferred.

Comment cleanup after learning: short matched trial

The earlier 2,048-step scratch experiment cannot establish whether removing comments helps an already trained model. This experiment compares two 1,024-update continuations from the same verified 72k numeric-weight-2 parent. Control retains original targets. Comments removes comments and empty lines outside strings only when parsing and compilation succeed and the Python syntax tree is identical. Each original target is replaced once; no examples or alternate solutions are added. Strings, docstrings, variable names, imports and executable statements are preserved.

Targets	Pilot /245	Confirm /493	Failures	Public /960
control	139	286	38	575
comments	142	286	41	560

Both arms keep the original tokenizer, architecture, optimizer state, seed 42, batch 128, numeric-token weight 2, learning rate 3e-5, 50-step warmup and strict greedy decoding. Equal updates and complete batches preserve the example budget; target lengths change. Held-out references remain original. Shortlist: none. A candidate needs positive gains on both held-out subsets with no extra total execution failures, followed by another seed before any longer run. Public scores do not determine this gate.

Thirteen focused tests passed. Verified parent/data CPU checks and the prior cleanup audit were reused; no additional CPU Colab runtime was allocated. Source, launcher and readiness records were saved to Drive and downloaded back before GPU allocation. Every checkpoint and exact test prediction was exported from Drive and verified before bot evaluation. The updated finalizer saves local diagnostic logs and requests release even if Drive fails.

Do not extend comment cleanup. The trained-model cleanup arm scores 560/960 (58.33%) versus 575/960 (59.90%) for its matched unchanged-target control: 15 fewer public answers. It gains 3 agreements on the first held-out subset, ties the second and adds 3 total execution failures, so it fails the preregistered extension gate. Together with the earlier scratch pilot, these two settings provide no evidence for longer comment-cleanup training. This does not prove every possible cleanup schedule would fail. Keep the existing best control checkpoint at 59.90%; current public rank remains 9/16 at 15:08 (+03:00).

The two-arm driver took 286.86 seconds (4.78 minutes), excluding allocation, Drive authentication and launcher package setup. Median GPU activity was 95%, mean 56.51%, peak memory 25,901 MiB. At 1,000 added updates, the training log reports 86.0 seconds for control and 98.7 seconds for cleanup; on-the-fly cleanup did not improve training speed. Colab showed no active sessions after completion. T4 timing remains unverified and deferred.

Sequence limits: CPU-only inference test

A CPU audit of the best checkpoint tokenizer found only 3 of 1,919 test questions longer than the 128-token input limit (maximum 166). The first 1,024 held-out questions include 2 above this limit, and 5 reference programs exceed the 192-token target limit. In the exact scored best export, 21 generated sequences lack an end-of-sequence token. One row overlaps, giving 23 affected rows.

Keep the same 42.48M model weights. Regenerate exactly those 23 rows on CPU with a 192-token input limit, a 384-token output limit, FP32, batch 8 and strict greedy decoding. Preserve the exact prior model-generated tokens on all other 1,896 rows. No code strings are edited, and no answer quality determines which result is retained. The full 1,919-row candidate is saved to Drive, exported and checked against generated token IDs before bot evaluation. This is a decoding test, with no training or GPU allocation.

Longer limits score 575/960 (59.90%), tying the retained best submission. They change 21 of 1,919 programs, but 5 of the 23 regenerated rows still lack EOS after the larger budget. Keep the existing inference notebook and checkpoint: this test shows no public score gain. Public rank is 9/16 at 15:25 (+03:00). No further extension is justified by this result.

The CPU driver took 44.31 seconds, including 19.72 seconds regenerating 23 rows; this excludes allocation, authentication and package setup. No GPU was allocated and no training was performed. A late monitoring request briefly reconnected a new CPU runtime after automatic release; that runtime was manually released, and Manage Sessions showed no active sessions. Incremental CPU timing is not a full 1,919-row inference measurement. T4 timing remains deferred and unverified.

Four focused checks passed. Model and tokenizer bytes are verified before CPU model loading. Each regenerated row is chosen solely from question length or missing EOS in the prior model output; no public reference answers enter the procedure. Source: 65f74d50bd9f6d68da0a555929783d0273a7d266. Evidence: experiments/sequence-length-audit-v1 and experiments/length-cap-cpu-v1.

Decoder-input dropout experiment

A 15.20-second local CPU diagnostic used the existing best checkpoint and 256 reserved training examples. With correct previous target tokens, number/operator-piece prediction accuracy was 85.35%; pairing each target with another reserved question reduced it to 37.70%. This shows substantial question dependence. Removing all preceding-code embeddings reduced the same accuracy to 0.05%, but the interventions differ in severity: these numbers do not measure causal contribution shares or establish that dropout will help.

Test three matched continuations from the verified 72,000-step model: unchanged control, 10% and 20% whole-token decoder embedding dropout. Each adds 1,024 updates at batch 128, numeric-token loss weight 2 and peak learning rate 3e-5. Keep every supplied question and reference target unchanged. Only training-time decoder embeddings are randomly zeroed, with retained embeddings scaled to preserve their expectation. Preserve start/end/padding embeddings and use a separate random generator. Inference stays strict greedy with dropout disabled.

Reject both dropout rates for extension. Control scores 575/960 (59.90%), 10% dropout 546/960 (56.87%) and 20% dropout 542/960 (56.46%). Held-out agreement drops by 13 and 16 of 738 usable references, while execution failures rise by 18 and 21. Neither arm passes the predefined gate. Keep the existing best checkpoint and inference notebook at 59.90%, rank 9/16 as reported at 15:44 (+03:00). This short continuation result does not establish whether training with dropout from scratch would help.

The three-arm driver took 393.35 seconds (6.56 minutes), excluding allocation, Drive authentication and package setup. Median GPU utilization was 97%, mean 65.60%, peak memory 23,885 MiB. All checkpoints and exact greedy predictions were saved to Drive before local verification. The control exactly matches a previously scored JSONL, so its 575/960 receipt was reused without a duplicate submission. Colab Manage Sessions visibly showed no active sessions after automatic release. T4 timing remains deferred and unverified.

Five unittest checks preceded launch, covering dropout and continuation behavior. Two runtime-finalization checks were subsequently rerun explicitly with pytest (seven dropout/continuation/runtime checks in total). Parent optimizer/data preflight was reused with checksum verification. Inspiration: Zhang et al., Token Drop mechanism for Neural Machine Translation, COLING 2020, https://aclanthology.org/2020.coling-main.379/. Only whole-token embedding dropout is tested; no extra targets, teacher or auxiliary objective is adopted. Source: a4a79fb19b7a45ba1bcec3517309d11cfe1d8b02. Evidence: experiments/conditioning-audit-cpu-v1 and experiments/decoder-dropout-1k-v1.

Learned source copying: CPU pilot

A 9.54-second CPU diagnostic on 256 reserved original examples found 207 number-piece prediction errors. The exact target token occurred in the question for 62 errors across 41 examples; an untrained average of all attention heads recovered 21 of those 62. These teacher-forced token diagnostics are not whole-program accuracy or a guaranteed achievable gain. Number pieces also include comments and strings.

Keep all 42,481,152 parent Transformer parameters frozen. Fit 545 parameters: a decoder-state copying gate and a learned mixture of 32 cross-attention heads. At every step, mix normal vocabulary probabilities with probabilities of copying existing question token IDs. Sum repeated source tokens, exclude padding/end tokens, and take the strict greedy maximum. No program repair, hand-written answer rule, generated target or pretrained component is used.

Use 4,096 original examples from supplied training shard zero, excluding all questions in the first 1,024 held-out rows. Fit for 512 small updates with the existing numeric/operator weight 2. Save and checksum-verify the custom CopyT5 checkpoint on Drive before generation, then reload the identical local weights; verify that every base parameter is unchanged. Compare copying enabled versus disabled on the same saved model and held-out subsets. Export all 1,919 test predictions in official order for bot evaluation.

Do not extend this frozen-model copying pilot. Control and learned copying both score 139/245 and 286/493 on the held-out subsets, with 40 total execution failures. The bot scores the new unique output 575/960 (59.90%), tying the simpler retained best. Public rank is 9/16 at 16:16 (+03:00). The positive-gain requirement fails; no second seed or long run is started. This tests a small frozen-feature adapter, not joint training of a Transformer and copy mechanism.

The CPU-only driver took 572.00 seconds (9.53 minutes), excluding allocation, Drive authorization and package setup. Fitting 545 parameters on cached frozen-model features took 1.93 seconds; all-1919 test generation took 179.83 seconds on CPU. Base weights remained identical. All checkpoints and predictions were saved and checksum-verified on Drive before local export and bot submission. The driver exited successfully; Colab showed no active sessions. No GPU was used, and T4 timing remains unverified.

Eleven focused pytest checks passed, including cached greedy generation, save/reload, copying normalization, gradient flow, disabled equivalence and runtime finalization. Real-checkpoint CPU preflight confirmed finite adapter gradients and no base gradients. Source: 0938a6265db552fd794e8e691cb30d646069ea60. Evidence: experiments/copyability-audit-cpu-v1 and experiments/copy-adapter-cpu-v1. Pointer-generator reference: See, Liu and Manning (2017), https://aclanthology.org/P17-1099/. Reused validation is exploratory; no long run is automatic.